

PRACTICAL-3

Problem Defination:

- Build a Node/Express backend for a Task Management system.
- Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet).
- Apply a request logging middleware that logs method, URL, and timestamp for every incoming request.
- Apply a global error handling middleware as the last middleware in the pipeline.
- Use correct HTTP status codes (200, 201, 404, 500) for each response.

GitHub: https://github.com/24dit061-Satyarajsinh/24DIT061_ITUE301-AWD

OUTPUT:

Install npm

```
C:\SEM5\AWD\task-manager-app\task-manager-app\backend>npm install
added 99 packages, and audited 100 packages in 6s
22 packages are looking for funding
  run `npm fund` for details
```

Start Backend

```
C:\SEM5\AWD\task-manager-app\task-manager-app\backend>npm start
> task-manager-api-backend@1.0.0 start
> node server.js

Server running on http://localhost:5000
GET /api/tasks/ - 2026-09-18T03:54:51.883Z
GET /api/tasks/ - 2026-09-18T03:54:51.894Z
PUT /api/tasks/3 - 2026-09-18T03:58:12.145Z
PUT /api/tasks/2 - 2026-09-18T03:58:35.659Z
POST /api/tasks/ - 2026-09-18T03:59:06.555Z
PUT /api/tasks/4 - 2026-09-18T03:59:30.418Z
PUT /api/tasks/3 - 2026-09-18T03:59:35.085Z
```

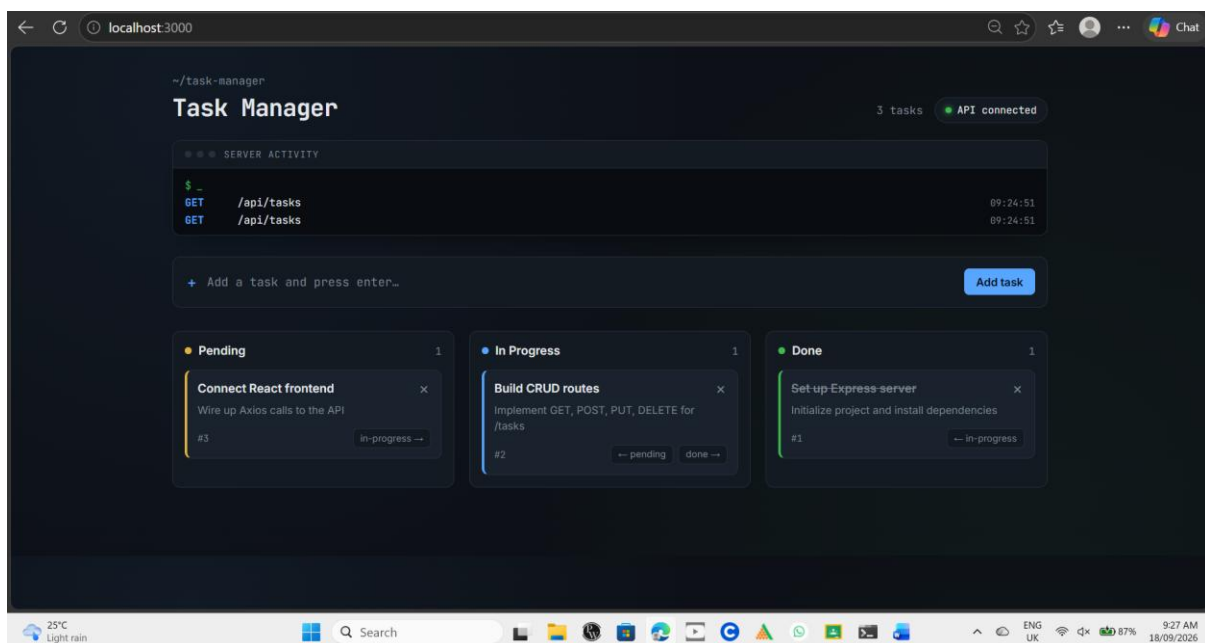
Strat Frontend

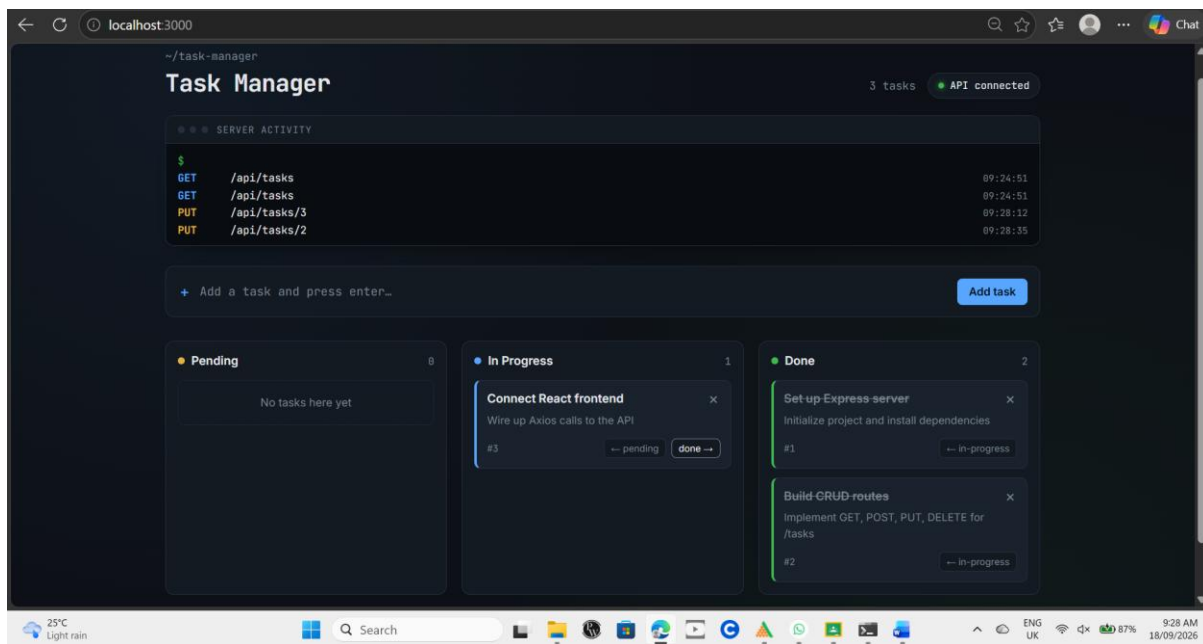
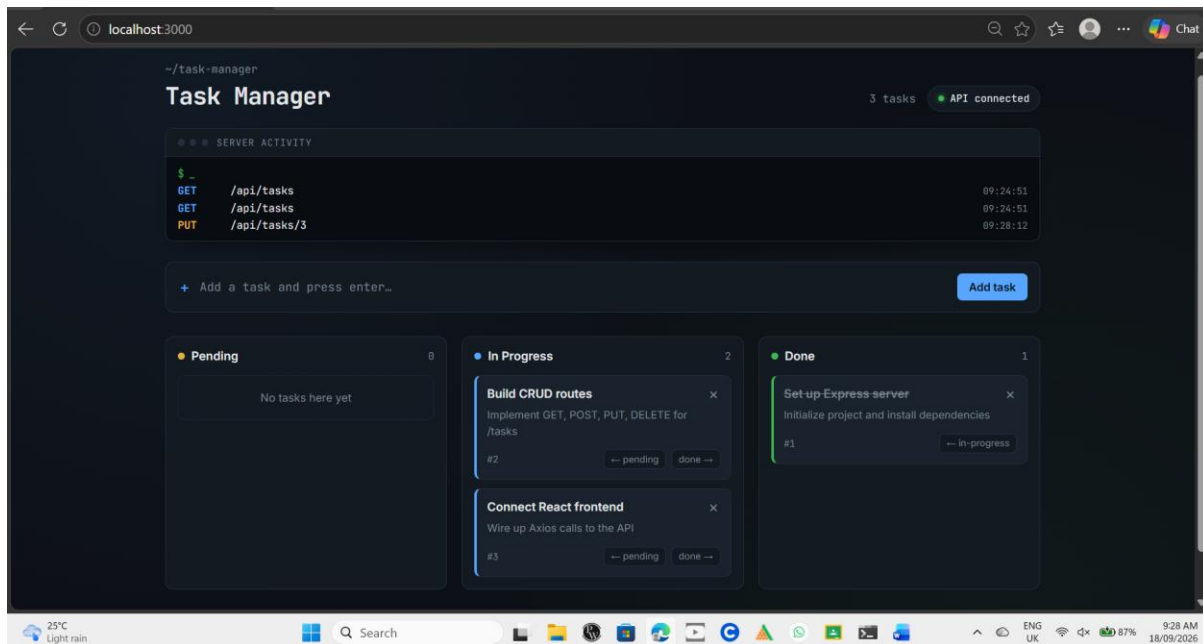
```
C:\SEM5\AWD\task-manager-app\task-manager-app\frontend>npm run dev

> task-manager-frontend@1.0.0 dev
> vite

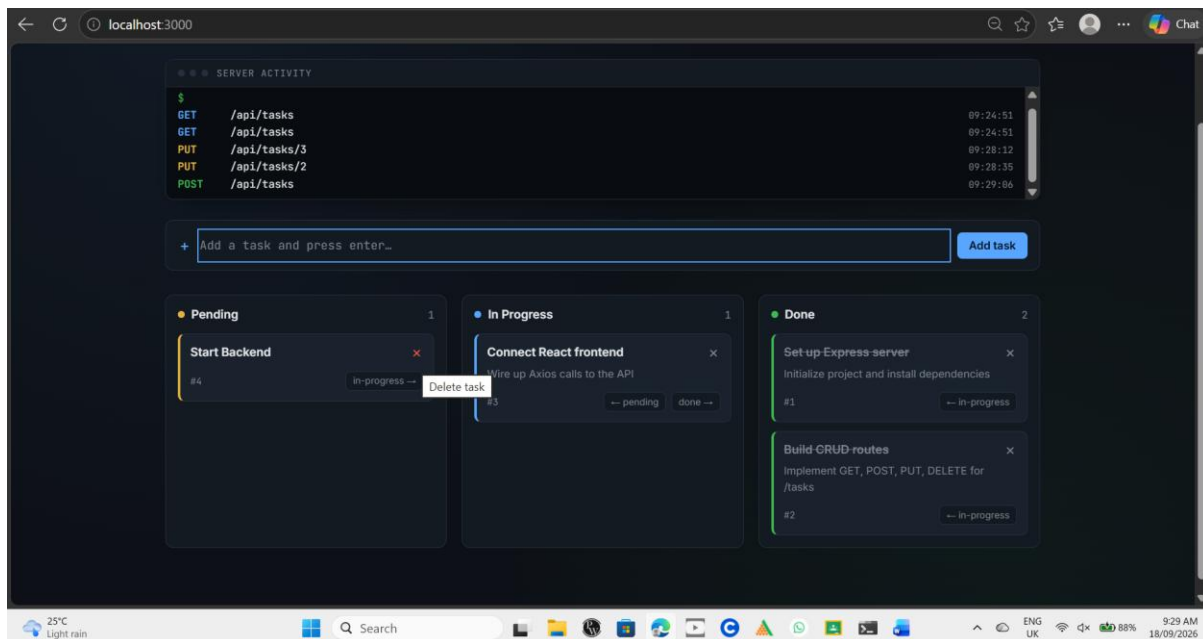
VITE v5.4.21 ready in 907 ms

➔ Local:   http://localhost:3000/
➔ Network: use --host to expose
➔ press h + enter to show help
```





Added New Tasks:



Key-Questions:

- Question:** Why must the error handling middleware be defined last in the middleware chain?

Answer: Error handling middleware is defined last because Express executes middleware in order. It must come after all routes and other middleware so that it can catch errors passed using `next(error)` and send an appropriate response to the client.
- Question:** What is the difference between `app.use()` and a route-specific middleware?

Answer: `app.use()` is generally used to apply middleware globally or to a specific path, whereas route-specific middleware is applied only to a particular route. For example, logging can use `app.use()`, while authentication can be applied only to a specific route.
- Question:** Why is it considered bad practice to send raw error stack traces to the client?

Answer: Raw error stack traces may expose sensitive information such as file paths, code structure, and internal implementation details. This can create security risks. Detailed errors should be logged on the server, while the client should receive a simple and safe error message.

Conclusion: The Task Management backend was successfully implemented using **Node.js** and **Express.js**. REST APIs were created for **creating, reading, updating, and deleting tasks** using an in-memory array. Request logging middleware was used to record the **HTTP method, URL, and timestamp** of each request. A global error-handling middleware was also implemented to handle errors and return appropriate HTTP status codes such as **200, 201, 404, and 500**.